Discord Data Platform: Petabyte-Scale Architecture

Cassandra to ScyllaDB Migration, Time-Window Bucketing & Real-Time Kafka

1. Storing Trillions of Messages: The Discord Problem Space

Discord connects hundreds of millions of users across massive real-time communication guilds. As users chat, send reactions, and transition voice channels, the system ingests petabytes of data:

The Operational Scale

- **Trillions of Messages:** Distributed across hundreds of physical nodes, growing by billions of records each week.
- **Read Access Pattern:** Highly skewed towards recent messages. When opening a channel, clients fetch the latest 50 messages, followed by backward cursor pagination.
- **Strict SLA:** Sub-15ms p99 read latencies on recent messages to guarantee smooth, seamless user scrolling.

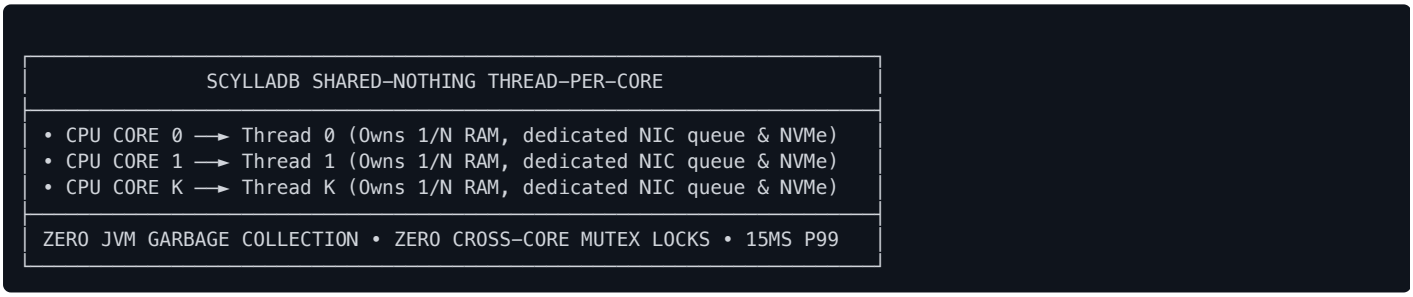
2. The Cassandra Wall: JVM Garbage Collection & Compactions

After outgrowing MongoDB, Discord initially migrated to Apache Cassandra. Cassandra scaled writes linearly, but at petabyte scale, it encountered critical operational limits:

- **Stop-the-World GC Pauses:** Java heap allocations during large partition reads triggered garbage collection stalls of 2–5 seconds, causing gateway request timeouts.
- **Compaction I/O Stalls:** Heavy LSM compaction tasks saturated NVMe disk queues, inflating p99 read latency beyond 1,000ms during peak gaming hours.
- **Unbounded Partitions:** High-traffic channels accumulated gigabytes of data under a single `channel_id`, overwhelming single Cassandra nodes.

3. Migrating to ScyllaDB: The C++ Seastar Engine

To eliminate JVM garbage collection permanently, Discord migrated to **ScyllaDB**—a C++ reimplementation of Cassandra built on the **Seastar asynchronous framework**:



4. Time-Window Bucketing: Capping Partition Size

To prevent channels from growing indefinitely into wide-partition bottlenecks, Discord partitions messages by a composite key combining channel ID and a temporal bucket:

```
CREATE TABLE messages (
    channel_id bigint,
    bucket int,                 -- Represents ~10 days of temporal message history
    message_id bigint,          -- Twitter Snowflake ID (encodes millisecond timestamp)
    author_id bigint,
    content text,
    PRIMARY KEY ((channel_id, bucket), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
```

Architectural Benefits of Time Bucketing

- **Bounded Partitions:** Even for mega-channels with millions of messages, each partition is strictly capped at ~100MB, eliminating compaction stalls.
- **Sequential Clustered I/O:** Sorting by `message_id DESC` leverages Snowflake timestamp ordering, fetching the latest 50 messages via contiguous disk reads.

5. Real-Time Kafka Streaming Pipeline

User actions, presence changes, and guild interactions stream through a centralized high-throughput Kafka backbone:

- **Gateway Ingestion:** WebSocket gateway nodes batch and publish hundreds of gigabytes per second of raw events to partitioned Kafka topics.
- **Secor S3/GCS Sink:** Real-time streaming consumers batch Kafka events and flush them to Google Cloud Storage (GCS) in Snappy-compressed Parquet format.
- **Analytical Isolation:** Product analytics and BI queries run against BigQuery external tables, preventing analytical workloads from degrading transactional ScyllaDB throughput.

6. Cassandra vs. ScyllaDB Performance Comparison

Metric / Vector	Apache Cassandra (Java)	ScyllaDB (C++ Seastar)
Runtime Environment	JVM (Garbage Collected)	Native C++ (Direct Memory Management)
p99 Read Latency	1,000ms – 5,000ms (GC Spikes)	**Sub-15ms (Predictable)**
Concurrency Architecture	Multi-threaded with mutex locks	Shared-nothing, Thread-per-core
Server Footprint	Requires 3x more nodes to absorb GC	Consolidated cluster (60% server reduction)

7. Real-Time Trust & Safety Anti-Abuse Pipelines

Operating massive public communities demands real-time spam detection and malware blocking before malicious content spreads:

- **Stateful Stream Classification:** Apache Flink and Rust microservices consume the live Kafka stream, maintaining sliding-window state to track message frequency, mention spam, and invite-link cascades per user.
- **Perceptual Hashing & ML Scoring:** Image and text payloads are hashed and scored against reputation models in under 50 milliseconds, quarantining rogue bots automatically.

8. Key Architectural Takeaways for Petabyte Data Platforms

Core Tenets Distilled from Discord's Architecture

- **Thread-Per-Core Eliminates GC at Scale:** In ultra-high-throughput NoSQL databases, Java Garbage Collection becomes an unpredictable operational liability. Shared-nothing C++ engines deliver rock-solid sub-15ms p99 SLAs.
- **Slice Partitions by Time Buckets:** Never let primary partitions grow unbounded. Adding a temporal bucket to partition keys caps maximum shard size and protects LSM-trees from compaction degradation.
- **Decouple Live Transactional Stores from Analytics:** Write live conversational state to high-speed NoSQL stores (ScyllaDB), while streaming raw events through Kafka into columnar cloud storage (Parquet + BigQuery) for analytics.
- **Leverage Snowflake IDs for Temporal Indexing:** Embedding millisecond timestamps into 64-bit ID generators eliminates secondary index overhead for time-ordered queries.